

🌟 DAY - 17 : Introduction to the DOM 🌟

Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#) 📁 | [X: @_19_neeraj](#) 🐦 | [GitHub: Neeraj05042001](#) 🧑

📌 What is DOM?

The **Document Object Model (DOM)** is a programming interface for web documents. It represents the structure of HTML and XML documents as a **tree of objects** that can be manipulated with JavaScript.

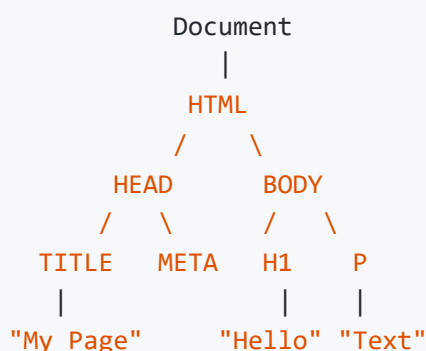
💡 **Key Concept:** Think of the DOM as a bridge that allows JavaScript to interact with HTML elements on your webpage.

JavaScript, allowing you to:

- Access elements on a webpage
- Modify content dynamically
- Change styles and attributes
- Respond to user interactions

🔍 Visualizing the DOM Tree

When a web page loads, the browser creates a DOM of the page, which is structured like a tree:



💡 **Key Point:** The DOM is not part of JavaScript; it's a separate API that browsers provide, which JavaScript can interact with.

🧐 Knowledge Check

- What does DOM stand for?
▶ Answer

- Why do we need the DOM in web development?
 - ▶ Answer
- What is the relationship between HTML and the DOM?
 - ▶ Answer

DOM and JavaScript

JavaScript and the DOM work together to create dynamic web pages:

- **HTML** creates the static structure of your webpage
- **CSS** styles that structure
- **JavaScript** manipulates the DOM to add dynamic behavior

This interaction allows you to:

- Change HTML content and attributes
- Modify CSS styles
- React to user events (clicks, keypresses, etc.)
- Create and remove elements

 **Important:** The DOM is not part of JavaScript! It's a separate Web API provided by the browser that JavaScript can interact with.

Knowledge Check

- If you change an HTML element using JavaScript, does it change the original HTML file?
 - ▶ Answer
- Name two things you can do with JavaScript and the DOM.
 - ▶ Answer

DOM Types

There are several important object types in the DOM hierarchy:

1. **Document**  : Represents the entire webpage and is the root node of the DOM tree.

```
console.log(document);
```

2. **Node**  : A generic term for any object in the DOM tree. Types include:

- Element nodes (HTML tags)
- Text nodes (text content)
- Attribute nodes (attributes of elements)

3. **Element** 🏠: A specific type of node representing HTML tags/elements.

4. **NodeList** 📋: An array-like collection of nodes.

5. **Attr** 🖋️: Represents an attribute of an element node.

```
<!-- src and alt are attributes -->

```

6. **NamedNodeMap** 🗃️: A collection of attribute nodes.

```
// Example showing different DOM types
console.log(document);           // Document
console.log(document.body);      // Element
console.log(document.body.childNodes); // NodeList
console.log(document.body.attributes); // NamedNodeMap
console.log(document.body.getAttribute('id')); // Attr value
```

💡 **Key Point:** Understanding these DOM types helps you navigate and manipulate the webpage efficiently.

🤔 Knowledge Check

- What's the difference between a Node and an Element?
 - ▶ Answer
- Is a paragraph (p) tag a Node or an Element? (Trick question!)
 - ▶ Answer

📌 Accessing DOM Elements

JavaScript provides several methods to access elements in the DOM:

1. **getElementById** 🎯

Used to find a single element with a specific ID attribute.

```
// HTML: <h1 id="heading">Welcome to the Day 17</h1>
let titleElem = document.getElementById("heading");
```

```
console.log(titleElem); // Returns the h1 element
```

- Returns a single element or `null` if not found
- IDs should be unique on a webpage
- Fastest way to access a specific element

2. `getElementsByClassName`

Used to find elements with a specific class name.

Returns a live `HTMLCollection` of elements with the specified class name.

```
// HTML: <p class="info">Hope you are enjoying 40 days of JavaScript!</p>
let infoElems = document.getElementsByClassName("info");
console.log(infoElems); // HTMLCollection of elements with class "info"
console.log(infoElems[0]); // First element with class "info"
```

- Returns an **HTMLCollection** (array-like object)
- Live collection (updates automatically if DOM changes)
- Access elements using index notation (`[0]` , `[1]` , etc.)

 **Remember:** `getElementsByClassName` returns a collection, not a single element!

To iterate through an `HTMLCollection`, you can convert it to an array:

```
[...infoElems].forEach((elem) => {
  console.log(elem);
});
```

3. `getElementsByTagName`

Used to find elements by their HTML tag name.

Returns a collection of elements with the specified tag name.

```
let pTagElems = document.getElementsByTagName("p");
console.log(pTagElems); // All <p> elements
```

- Returns an `HTMLCollection` of elements
- Live collection (updates automatically)
- Useful for selecting all instances of a particular HTML tag

4. `querySelector`

Used to find the first element that matches a CSS selector.

Returns the first element that matches a specified CSS selector.

```
// Find the first paragraph with class "info"
let para = document.querySelector("p.info");
console.log(para); // Returns the first p element with class "info"

// Find element by ID using CSS selector syntax
let hOne = document.querySelector("#heading");
console.log(hOne); // Returns the element with id "heading"
```

- Returns the first matching element or `null` if none found
- Uses CSS selector syntax (very versatile)
- Can use complex selectors like `"div > p.highlight"`

5. `querySelectorAll` 🔍

Similar to `querySelector` but returns all matching elements.

```
// Find all paragraphs with class "info"
let paras = document.querySelectorAll("p.info");
console.log(paras); // Returns NodeList of all p elements with class "info"
```

- Returns a **NodeList** containing all matching elements
- Static collection (doesn't update automatically)
- Can iterate using `forEach` directly:

```
paras.forEach(para => {
  console.log(para.textContent);
});
```

💡 **Tip:** `querySelector` and `querySelectorAll` use CSS selector syntax, making them very powerful!

Summary of DOM Access Methods:

Method	Returns	Live Collection?	Syntax
<code>getElementById</code>	Single Element	N/A	<code>document.getElementById("id")</code>
<code>getElementsByClassName</code>	HTMLCollection	Yes	<code>document.getElementsByClassName("ta</code>
<code>getElementsByTagName</code>	HTMLCollection	Yes	<code>document.getElementsByTagName("ta</code>

Method	Returns	Live Collection?	Syntax
querySelector	Single Element	N/A	<code>document.querySelector("css-selector")</code>
querySelectorAll	NodeList	No	<code>document.querySelectorAll("css-selector")</code>

💡 **Key Point:** `querySelector` and `querySelectorAll` are more versatile but slightly slower than the specific methods.

✅ Quick Check:

If you needed to select all paragraphs with a class of "highlight" inside a div with ID "content", which method would you use and what would the code look like?



🤔 Knowledge Check

- What's the difference between `querySelector` and `getElementById`?
▶ Answer
- If there are three elements with the class "info", what will `querySelector("p.info")` return?
▶ Answer
- What's the difference between an `HTMLCollection` and a `NodeList`?
▶ Answer

📌 Manipulating the DOM

Once you've accessed elements, you can manipulate them in various ways:

1. Change content:

```
// change text content
element.textContent = "New text";

// Change HTML content (can include tags)
element.innerHTML = "<strong>Bold text</strong>";
```

2. Modify attributes:

```
// Get attribute
let imgSrc = img.getAttribute("src");

// Set attribute
img.setAttribute("alt", "Description");

// Check if attribute exists
let hasClass = element.hasAttribute("class");

// Remove attribute
element.removeAttribute("style");
```

3. Change styling:

```
// Direct style modification
element.style.backgroundColor = "yellow";
element.style.fontSize = "20px";
```

4. Add/remove classes:

```
element.classList.add("highlight");
element.classList.remove("hidden");
element.classList.toggle("active");
```

5. Creating and Adding Elements

```
// Create a new element
let newParagraph = document.createElement("p");

// Add content to it
newParagraph.textContent = "This is a new paragraph";

// Add it to the document
document.body.appendChild(newParagraph);
```

💡 **Key Point:** Always try to minimize direct DOM manipulation as it can be performance-intensive.

✅ Quick Check:

How would you create a new button element with text "Click me" and add it to a div with id "container"?

Mini Projects

Mini Project 1: Text Highlighter

This project highlights paragraphs with the class "info" when a button is clicked.

```
function highlightText() {  
  console.log("About to highlight a text...");  
  let elements = document.querySelectorAll("p.info");  
  elements.forEach((element) => {  
    element.style.backgroundColor = "yellow";  
  });  
}
```

HTML for reference:

```
<h1 id="heading">Welcome to the Day 17</h1>  
<p class="info">Hope you are enjoying 40 days of JavaScript!</p>  
<p class="info">Make sure to Subscribe to tapaScript!</p>  
<p>Hope you are enjoying it!</p>  
<button onclick="highlightText()">Highlight</button>
```

Mini Project 2: Search Filter

This project filters a list of items as the user types in a search box.

```
function filterList() {  
  const inputElem = document.getElementById("searchInput");  
  const input = inputElem.value;  
  const items = document.querySelectorAll("ul#itemList li");  
  
  items.forEach((item) => {  
    item.style.display = item.innerText.toLowerCase().includes(input.toLowerCase())  
      ? "block"  
      : "none";  
  });  
}
```

HTML for reference:

```
<input type="text" id="searchInput" placeholder="Search..." onkeyup="filterList()" />  
<ul id="itemList">  
  <li>Apple</li>  
  <li>Banana</li>  
  <li>Cherry</li>  
  <li>Grapes</li>  
  <li>Orange</li>  
</ul>
```


Which DOM Method to Use When

Choosing the right DOM method can make your code more efficient and readable. Here's a guide on when to use each method:

1. `getElementById`

Best for: Accessing a single, unique element with a known ID.

```
document.getElementById("uniqueElementId");
```

When to use: When you have assigned a unique ID to an element and need quick access.

Advantages: Fastest method, very specific, clear intent. **Limitations:** Only works with IDs, returns null if not found.

2. `getElementsByClassName`

Best for: Accessing multiple elements that share a class.

```
document.getElementsByClassName("common-class");
```

When to use: When you need all elements with a specific class and might need to track DOM changes automatically. **Advantages:** Returns a live collection that updates automatically when DOM changes. **Limitations:** Returns HTMLCollection (no `forEach`), requires conversion to array for many operations.

3. `getElementsByTagName`

Best for: Accessing all elements of a specific type.

```
document.getElementsByTagName("div");
```

When to use: When you need all elements of a specific tag type (e.g., all paragraphs, all divs).

Advantages: Returns a live collection, good for broad selections. **Limitations:** Not specific enough for many use cases, returns HTMLCollection.

4. `querySelector`

Best for: Finding the first element that matches specific criteria.

```
document.querySelector("#id");  
document.querySelector(".class");
```

```
document.querySelector("div.container > p");
```

When to use: When you need precise selection using CSS selectors or when the selection might match multiple elements but you only need the first. **Advantages:** Very flexible, accepts any valid CSS selector, clear syntax. **Limitations:** Only returns the first matching element, slightly slower than direct methods like `getElementById`.

5. `querySelectorAll` 🔍

Best for: Finding all elements matching specific criteria.

```
document.querySelectorAll("p.highlight");
```

When to use: When you need all elements matching complex criteria or when you plan to iterate through the results. **Advantages:** Very flexible, returns a `NodeList` with `forEach` method, accepts any valid CSS selector. **Limitations:** Returns a static `NodeList` (doesn't auto-update), slightly slower than direct methods.

Deciding Factors Quick Guide:

If you need to...	Use this method
Find a single element by ID	<code>getElementById</code>
Find multiple elements by class	<code>getElementsByClassName</code>
Find multiple elements by tag name	<code>getElementsByTagName</code>
Find first element with complex criteria	<code>querySelector</code>
Find all elements with complex criteria	<code>querySelectorAll</code>
Need a live collection (auto-updating)	<code>getElementsByClassName/TagName</code>
Need to iterate through results easily	<code>querySelectorAll</code>

💡 Best Practices

1. **For performance:** If you only need an element by ID, use `getElementById` as it's the fastest.
2. **For flexibility:** `querySelector`/`querySelectorAll` are the most versatile and modern approach.
3. **For code readability:** Choose methods that make your intention clear:

```
// Good - intent is clear
document.getElementById("main-header");
```

```
// Less clear - using querySelector for ID
document.querySelector("#main-header");
```

4. **For dynamic DOM:** If you need a collection that automatically updates when the DOM changes, use `getElementsByClassName` or `getElementsByTagName`.
 5. **For complex selections:** Use `querySelector`/`querySelectorAll` with specific CSS selectors rather than getting all elements and filtering them with JavaScript.
- The DOM is a tree-like representation of your HTML document that JavaScript can interact with.
 - DOM manipulation allows you to create dynamic, interactive web pages.
 - There are multiple ways to access elements: by ID, class, tag name, or CSS selectors.
 - The browser's DevTools are invaluable for inspecting and debugging DOM-related code.
 - Understanding the DOM is fundamental to front-end web development.

Common DOM Events and Event Handling

Understanding DOM events is crucial for creating interactive web applications.

Common DOM Events

1. Mouse Events:

- `click` : When an element is clicked
- `dblclick` : When an element is double-clicked
- `mouseenter` / `mouseleave` : When mouse enters/leaves an element
- `mouseover` / `mouseout` : Similar to enter/leave but bubbles (affects children)
- `mousedown` / `mouseup` : When mouse button is pressed/released

2. Keyboard Events:

- `keydown` : When a key is pressed down
- `keyup` : When a key is released
- `keypress` : When a key that produces a character is pressed (deprecated)

3. Form Events:

- `submit` : When a form is submitted
- `change` : When form control's value changes (after losing focus)
- `input` : When form control's value changes (immediately)
- `focus` / `blur` : When an element gains/loses focus

4. Document/Window Events:

- `load` : When page has fully loaded
- `resize` : When window is resized
- `scroll` : When document or element is scrolled
- `DOMContentLoaded` : When HTML is loaded and parsed (before images, etc.)

Adding Event Listeners

1. Inline HTML attribute (least recommended):

```
<button onclick="doSomething()">Click me</button>
```

2. DOM property (simple but limited):

```
element.onclick = function() {  
  // do something  
};
```

3. `addEventListener` (recommended):

```
element.addEventListener("click", function(event) {  
  // do something  
  console.log(event.target);  
});
```



DevTools and DOM

Modern browsers provide powerful developer tools for inspecting and manipulating the DOM:

Using Browser DevTools to Explore the DOM

1. Open DevTools:

- Chrome/Edge: F12 or Right-click > Inspect
- Firefox: F12 or Right-click > Inspect Element
- Safari: Cmd+Opt+I (enable developer menu first)

2. The Elements/Inspector tab shows the DOM tree

- You can expand/collapse nodes
- Select elements on the page by clicking
- See computed styles and box model
- View event listeners attached to elements

3. The Console tab allows JavaScript interaction

- Test DOM manipulation commands directly
- Use `$()` as shorthand for `document.querySelector()`
- Use `$$()` as shorthand for `document.querySelectorAll()`

Common DevTools Actions

- **Edit HTML:** Double-click on elements in the Elements tab
- **Modify styles:** Use the Styles panel on the right
- **Debug JavaScript:** Set breakpoints in DOM event listeners
- **Monitor performance:** Check how DOM operations affect page speed

💡 **Key Point:** DevTools is your best friend for understanding and debugging DOM interactions.

✅ Quick Check:

How would you use DevTools to find out what CSS is causing a specific element to have a red border?

🔑 Key Takeaways

- The DOM is a tree-like representation of your HTML document that JavaScript can interact with.
 - DOM manipulation allows you to create dynamic, interactive web pages.
 - There are multiple ways to access elements: by ID, class, tag name, or CSS selectors.
 - Use `getElementById` for performance when targeting unique elements
 - Use `querySelector/querySelectorAll` for flexibility and complex selections
 - Use `getElementsByClassName/TagName` when you need live collections
 - DOM properties and methods allow you to:
 - Change content (`textContent` , `innerHTML`)
 - Modify styles and classes (`style` , `classList`)
 - Navigate the DOM tree (parent/child/sibling properties)
 - Create and modify elements
 - Events are how we make pages interactive:
 - Common events include `click` , `submit` , `input` , and `load`
 - `addEventListener` is the preferred way to handle events
 - Event delegation uses bubbling to handle events efficiently
 - The browser's DevTools are invaluable for inspecting and debugging DOM-related code.
 - Understanding the DOM is fundamental to front-end web development.
-

Remember: The DOM is the bridge between your static HTML and dynamic JavaScript functionality. Mastering DOM manipulation is a crucial skill for any JavaScript developer!

Practice Tasks

Now that you've learned about the DOM, let's apply your knowledge with these exercises:

Exercise 1: Element Selection

Write JavaScript code to:

1. Select the first paragraph on the page
2. Select all elements with class "highlight"
3. Select all images inside an article with id "main-content"
4. Select the third list item in an unordered list

Exercise 2: Content Manipulation

Given this HTML:

```
<div id="container">
  <h2 id="title">Original Title</h2>
  <p class="content">Some content here</p>
</div>
```

Write JavaScript to:

1. Change the title to "Updated Title"
2. Add an exclamation mark to the paragraph content
3. Create and append a new paragraph with text "New paragraph added dynamically"
4. Add a CSS class "highlighted" to the container div

Exercise 3: Event Handling

Create a button that, when clicked:

1. Changes its own text from "Click Me" to "Clicked!"
2. Changes its background color
3. Creates and displays a message "Button was clicked at [current time]"

Exercise 4: DOM Navigation

Given an element with id "target", write code to:

1. Access its parent element
2. Get its next sibling
3. Get its first child
4. Count how many children it has

Exercise 5: Practical Application

Create a simple image gallery:

1. Start with 5 image URLs in an array
2. Generate img elements dynamically and append them to a container
3. Add functionality so clicking on any image displays it larger in a preview area
4. Add Next/Previous buttons to navigate through the gallery

Exercise 6: DOM Challenge

Create a "theme switcher":

1. Create a button labeled "Toggle Dark Mode"
2. When clicked, it should:
 - Change the page background to dark and text to light (or vice versa)
 - Update button text accordingly
 - Store the user's preference in localStorage
3. On page load, check localStorage and apply the saved theme

Exercise 7: List Builder

Create an interface with:

1. An input field
2. An "Add" button
3. A list (initially empty)
4. When "Add" is clicked, the input text should become a new list item
5. Each list item should have a "Delete" button that removes it

Exercise 8: Form Validation

Create a simple form with:

1. Name field (required)
2. Email field (must contain @ and .)
3. Password field (minimum 8 characters)
4. Submit button

5. Use DOM manipulation to display error messages when validation fails

Exercise 9: Interactive Quiz

Create a 3-question quiz:

1. Generate questions and multiple-choice answers from a JavaScript object
2. When a user selects an answer, immediately show if it's correct or wrong
3. Keep score and display the final result when all questions are answered

Exercise 10: DOM Performance

Write a function that:

1. Creates 1000 div elements
2. Sets text content and a class on each
3. Appends them to the document

Then optimize it using document fragments and compare the performance.

💡 **Final Tip:** Remember that excessive DOM manipulation can slow down your page. Batch your changes when possible, and consider using tools like virtual DOM for complex applications.



Let's Connect & Level Up Together

Follow me for daily **JavaScript** tips, insightful notes, and project-based learning.



X (Twitter)



LinkedIn



Telegram



Threads



Dive into the full notes on GitHub → **40 Days JavaScript**

© 2025 • Crafted with ❤️ by Neeraj



Next: Day - 18: DOM Manipulations



Happy JavaScript coding!

